Build an Obstacle-Avoiding Rover

Arduino, ultrasonic sensor and servo

Yantrika: Robotics and Coding Club

RSA, IACS

Workshop

Today's plan (about 3 hours)

- Intro, how the rover thinks, parts check: 20 min
- Chassis assembly and motor mounting: 30 min
- Wiring motors, driver and power: 30 min
- Code step 1 (motors) and step 2 (sensor): 45 min
- Servo and full avoidance code: 35 min
- Tuning, challenges, wrap-up: 20 min

What you will learn

- A robot is a loop: **sense, think, act**
- Driving DC motors with an L298N and PWM
- Measuring distance with an HC-SR04
- Writing small, testable code and combining it step by step
- Debugging hardware and software separately

The rover loop

Sense	Think	Act
HC-SR04 measures the distance ahead. The servo turns the sensor left and right.	Is something closer than 20 cm? Which side has more free space?	Drive forward, stop, back up, or spin toward the clearer side.

Parts list

Qty	Part	Notes
1	Arduino Uno	Nano works with the same pins
1	L298N motor driver	Dual H-bridge
1	2WD or 4WD chassis kit	TT gear motors and wheels
1	HC-SR04 ultrasonic sensor	5 V version
1	SG90 micro servo	Holds the sensor
1	2×18650 holder (7.4 V)	Or 6×AA, with a slide switch
25	Jumper wires	Male-male and male-female

Before we start

- Install the Arduino IDE (`arduino.cc/en/software`)
- Tools $\rightarrow$ Board $\rightarrow$ Arduino Uno, then pick the right port
- The Servo library is already included
- Use fully charged batteries: weak cells cause random resets
- Work in pairs: one wires, one checks, then swap

PART 1: How it works

Ultrasonic distance, H-bridges and PWM

The HC-SR04 ultrasonic sensor

- TRIG sends a short burst of 40 kHz sound
- ECHO stays HIGH while the sound travels to the obstacle and back
- Longer echo means a farther object
- Works best on flat, hard surfaces
- Soft cloth or angled walls reflect poorly

Echo time to distance

$$d\,(\text{cm}) = \frac{t\,(\mu s)}{58}$$

Sound travels about 343 m/s. The echo covers the distance twice (there and back), so we halve it. That gives roughly 58 μs per centimetre.

The L298N H-bridge

IN1	IN2	Motor result
HIGH	LOW	Forward
LOW	HIGH	Backward
LOW	LOW	Stop (coast)
HIGH	HIGH	Brake

Speed with PWM

- The enable pins (ENA, ENB) take a PWM signal
- `analogWrite(pin, 0..255)` sets the speed
- Below about 90, most TT motors only hum and stall
- Start near 150 and tune from there
- Remove the ENA and ENB jumpers on the L298N

PART 2: Build it

Chassis, wiring and power

Chassis assembly

1. Mount the gear motors on the bottom plate
2. Attach the wheels; fit the caster on 2WD kits
3. Mount the battery holder and slide switch
4. Fix the L298N and Arduino on the top deck
5. Leave room to route wires

Sensor and servo mount

- Fix the servo at the front edge of the top deck
- Attach the HC-SR04 to the servo horn
- At 90° the sensor must face straight ahead
- Leave slack in the wires so it swings freely
- Tilt slightly down to catch low obstacles

Wiring: motors and driver

L298N pin	Connects to	Purpose
OUT1, OUT2	Left motor(s)	Left side
OUT3, OUT4	Right motor(s)	Right side
ENA	Arduino D5	Left speed (PWM)
IN1, IN2	Arduino D7, D8	Left direction
IN3, IN4	Arduino D9, D10	Right direction
ENB	Arduino D6	Right speed (PWM)

Wiring: sensor and servo

Part pin	Connects to
HC-SR04 VCC / GND	Arduino 5 V / GND
HC-SR04 TRIG	Arduino D12
HC-SR04 ECHO	Arduino D11
Servo red / brown	Arduino 5 V / GND
Servo orange (signal)	Arduino D3

Wiring: power

From	To
Battery + (via switch)	L298N 12 V terminal
Battery −	L298N GND terminal
L298N GND	Arduino GND (**common ground**)
L298N 12 V terminal	Arduino VIN (7.4 V is fine)

Safety and checkpoint

Safety

- Wire with the battery off
- Never run USB and battery together while motors are on
- Keep fingers clear of the wheels

Checkpoint

- All grounds joined
- ENA and ENB jumpers removed
- No wire on the wrong voltage
- Switch off before the first upload

PART 3: Code it

Three small steps. If something breaks, you know exactly where.

Step 1: motor test

```
const uint8_t ENA=5, IN1=7, IN2=8;   // left
const uint8_t ENB=6, IN3=9, IN4=10;  // right
const uint8_t SPEED = 150;

void setup() {
  for (uint8_t p : {ENA,IN1,IN2,ENB,IN3,IN4}) pinMode(p, OUTPUT);
}
void drive(bool lFwd, bool rFwd) {
  digitalWrite(IN1, lFwd); digitalWrite(IN2, !lFwd);
  digitalWrite(IN3, rFwd); digitalWrite(IN4, !rFwd);
  analogWrite(ENA, SPEED); analogWrite(ENB, SPEED);
}
void stopMotors() { analogWrite(ENA,0); analogWrite(ENB,0); }

void loop() {
  drive(true,true);   delay(2000); stopMotors(); delay(1000);
  drive(false,false); delay(2000); stopMotors(); delay(1000);
  drive(false,true);  delay(2000); stopMotors(); delay(1000);
  drive(true,false);  delay(2000); stopMotors(); delay(1000);
}
```

Lift the rover so the wheels spin in the air.

Step 1 checkpoint

- Forward: both sides must turn forward
- One side backward? Swap that motor's two wires on the L298N
- Spin left actually spins right? Swap the left and right pairs
- Nothing moves? Check the switch, battery and the L298N LED
- Motors hum only? Raise `SPEED`

Step 2: distance sensor test

```
const uint8_t TRIG = 12, ECHO = 11;

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
}
long readCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);  // 30 ms timeout
  if (us == 0) return 400;               // nothing detected
  return us / 58;
}
void loop() {
  Serial.print("Distance: ");
  Serial.print(readCm());
  Serial.println(" cm");
  delay(200);
}
```

Step 3: the avoidance logic

1. Cruise

Read distance (average of 3). Above 20 cm: drive forward.

2. React

Too close: stop and back up for 300 ms.

3. Scan

Servo to 160°, measure. Servo to 20°, measure. Return to 90°.

4. Turn

Both sides blocked: turn around. Else spin toward the larger distance.

Step 3a: sensing and motion helpers

```
#include <Servo.h>
// pins as before; SERVO_PIN = 3
const int CRUISE_SPEED=150, TURN_SPEED=170, STOP_CM=20;
const int TURN_MS_90=450;   // calibrate!
Servo head;

long readCmSmooth() {             // average of 3 readings
  long sum = 0;
  for (int i=0; i<3; i++) { sum += readCm(); delay(10); }
  return sum / 3;
}
long lookAt(int angle) {
  head.write(angle); delay(450);
  return readCmSmooth();
}
void setMotor(uint8_t en, uint8_t a, uint8_t b, int speed) {
  digitalWrite(a, speed > 0);
  digitalWrite(b, speed < 0);
  analogWrite(en, abs(speed));   // signed speed: -255..255
}
void drive(int l, int r) { setMotor(ENA,IN1,IN2,l); setMotor(ENB,IN3,IN4,r); }
void stopMotors() { drive(0,0); }
```

Step 3b: the main loop

```
void loop() {
  long ahead = readCmSmooth();
  if (ahead > STOP_CM) {
    drive(CRUISE_SPEED, CRUISE_SPEED);
    delay(50);
    return;
  }
  stopMotors();  delay(200);
  drive(-CRUISE_SPEED, -CRUISE_SPEED);  delay(300);
  stopMotors();

  long left  = lookAt(160);
  long right = lookAt(20);
  head.write(90);  delay(300);

  if (left < STOP_CM && right < STOP_CM) {
    drive(TURN_SPEED, -TURN_SPEED);  delay(TURN_MS_90 * 2);  // U-turn
  } else if (left > right) {
    drive(-TURN_SPEED, TURN_SPEED);  delay(TURN_MS_90);      // left
  } else {
    drive(TURN_SPEED, -TURN_SPEED);  delay(TURN_MS_90);      // right
  }
```

Tuning

Setting	What to adjust
`STOP_CM`	30 for a faster rover; 15 for tight spaces
`CRUISE_SPEED`	About 120 for accurate sensing; higher for speed
`TURN_MS_90`	Measure the real turn angle; depends on battery and floor
Servo angles	Re-seat the horn if 90° is not straight ahead

Troubleshooting

Problem	Likely fix
Arduino resets when motors start	Fresh batteries; add a 470 μF capacitor across the battery
Motors hum, no motion	Raise `CRUISE_SPEED` to 150 or more
One side never moves	Check ENA/ENB jumper removed; wires reach D5/D6
Distance always 400	TRIG and ECHO swapped, or loose 5 V / GND
Servo twitches	Power it from a separate 5 V supply, common ground

Challenges

- Slow down smoothly as an obstacle approaches
- Wall follower using the servo at 20°
- Buzzer when backing up, LED when turning
- Bluetooth override with an HC-05
- Cardboard maze: fastest escape wins
- Print scan data over Serial and plot the room

Where Yantrika goes next

- Add wheel encoders for distance and speed control
- Replace the single sensor with a ring or a lidar
- Try line following and a PID controller
- Move to ESP32 for Wi-Fi and Bluetooth control
- Aim for mapping and navigation projects

Thank you

Questions, demos and a rover maze run

Yantrika: Robotics and Coding Club